

Banknote Authentication Project - Machine Learning from Scratch

1. Project Title

Banknote Authentication: Comparison of Naive Bayes, KNN, and Decision Tree ID3 Implementations from Scratch

2. Author

sheyda fathi

3. Abstract

This project implements and compares three classical machine learning classifiers from scratch using only NumPy, pandas, and matplotlib. No machine learning libraries (scikit-learn, etc.) are used for algorithm implementation. The classifiers are:

1. Gaussian Naive Bayes
2. K-Nearest Neighbors (KNN) with $k = 1, 3, 5, 7$
3. Decision Tree ID3 with $\text{max_depth} = 5$

The goal is to classify banknotes as authentic (0) or forged (1) using four features extracted from wavelet-transformed images. Models are evaluated using 70/15/15 train/validation/test split, 5-fold cross-validation, paired t-tests for statistical significance, and learning curve analysis.

4. Dataset Information

Property	Value
Name	UCI Banknote Authentication Dataset
File	data_banknote_authentication.txt
Total Samples	1372
Features	4 continuous features
Classes	2 (0 = authentic, 1 = forged)
Class Distribution	762 authentic (55.5%), 610 forged (44.5%)

5. Feature Descriptions

Feature	Description
Variance	Variance of wavelet-transformed image
Skewness	Skewness of wavelet-transformed image
Kurtosis	Kurtosis of wavelet-transformed image
Entropy	Entropy of image

6. Project Structure

```
.
├── project.py # Main Python script (all code)
├── data_banknote_authentication.txt # Raw dataset
├── output/ # Generated plots folder
│   ├── learning_curves_comparison.png # All models comparison
│   ├── algorithm_comparison.png # Bar chart with error bars
│   ├── learning_curves_report.png # Clean learning curves for report
│   ├── learning_curve_naive_bayes.png # NB learning curve
│   ├── learning_curve_knn_(k=1).png # KNN learning curve
│   └── learning_curve_decision_tree.png # DT learning curve
└── README.txt # This file
```

7. Requirements

- Python 3.8 or higher
- NumPy
- pandas
- matplotlib
- scipy (only for t-test, allowed by project)

8. Installation

`pip install numpy pandas matplotlib scipy`

9. How to Run

1. Place the dataset file `data_banknote_authentication.txt` in the same directory as `project.py`
 2. Run the script:
bash
`python project.py`
 3. The script will:
 - Load and preprocess the data
 - Train all three algorithms
 - Perform 5-fold cross-validation
 - Run paired t-tests for statistical comparison
 - Generate learning curves
 - Save all plots to the `output/` folder
-

10. Methodology

1. Data Preparation

Step	Description
Data Loading	Load 1372 samples with 4 features + 1 label
Min-Max Normalization	Scale features to [0, 1] range for KNN
Train/Val/Test Split	70% training, 15% validation, 15% test
5-Fold Cross-Validation	For final evaluation and t-test

2. Min-Max Normalization

Formula used (implemented from scratch):

$$x_{\text{normalized}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$$

Parameters (min, max) are computed ONLY on training data to prevent data leakage.

3. Algorithm 1: Gaussian Naive Bayes

Assumptions:

- Features are independent given the class
- Features follow Gaussian (normal) distribution

Training:

For each class c and feature i :

- Calculate mean $\mu_{c,i}$
- Calculate variance $\sigma^2_{c,i}$
- Calculate prior $P(y=c)$

Prediction (using log-probabilities for stability):

$$\log P(y=c|x) = \log P(y=c) + \sum_i \log P(x_i|y=c)$$

$$\text{where } P(x_i|y=c) = (1/\sqrt{2\pi\sigma^2}) * \exp(-(x-\mu)^2/(2\sigma^2))$$

4. Algorithm 2: K-Nearest Neighbors (KNN)

Characteristics:

- Lazy learner (stores all training data)
- Distance metric: Euclidean distance
- Decision: Simple majority voting (no weights)

Distance Formula:

$$d(x, x') = \sqrt{(\sum_i (x_i - x'_i)^2)}$$

Hyperparameters tested: $k = 1, 3, 5, 7$

5. Algorithm 3: Decision Tree ID3

Feature Discretization:** Continuous features discretized using mean threshold

- Value $\leq$ mean $\rightarrow$ 0 (low)
- Value $>$ mean $\rightarrow$ 1 (high)

Entropy Formula:

$$H(S) = - \sum p(c) * \log_2(p(c))$$

Information Gain:

$$IG(S, A) = H(S) - \sum (|S_v|/|S|) * H(S_v)$$

Pre-pruning: max_depth = 5 (as specified in project)

6. Evaluation Metrics

All metrics implemented from scratch:

Metric	Formula
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$
Precision	$TP / (TP + FP)$
Recall	$TP / (TP + FN)$
F1 Score	$2 * (Precision * Recall) / (Precision + Recall)$

7. Statistical Comparison

Method: Paired t-test between model pairs

Null Hypothesis: No significant difference between model performances

Significance Level: $\alpha = 0.05$

If p-value < 0.05 , the difference is statistically significant.

8. Learning Curves

Training sizes tested: 20%, 40%, 60%, 80%, 100% of data

For each size:

- Perform 5-fold cross-validation
- Record mean accuracy and standard deviation

Interpretation:

- High Bias (Underfitting): Model improves significantly with more data
- High Variance (Overfitting): Large gap between training and validation
- Low Bias/Low Variance: Stable performance across data sizes

11.Results

Test Set Performance (15% Holdout)

Algorithm	Accuracy	Precision	Recall	F1 Score
Naive Bayes	~0.84	~0.85	~0.82	~0.83
KNN (best k)	~1.00	~1.00	~1.00	~1.00
Decision Tree (depth=5)	~0.95	~0.95	~0.94	~0.94

5-Fold Cross-Validation Results

Algorithm	Accuracy (mean ± std)
Naive Bayes	0.85 ± 0.02
KNN (best k)	0.998 ± 0.002
Decision Tree (depth=5)	0.94 ± 0.01

Paired t-test Results (p-values)

Comparison	p-value	Significant (p < 0.05)
Naive Bayes vs KNN	< 0.001	YES
Naive Bayes vs Decision Tree	< 0.001	YES
KNN vs Decision Tree	< 0.001	YES

Best Algorithm

KNN achieved the highest accuracy with:

- Best k value: determined by validation set
- Test set accuracy: ~1.000
- Lowest standard deviation across folds

12.Key Findings

1. KNN is the best performing algorithm** on this dataset. Its ability to capture complex decision boundaries without strong assumptions about data distribution gives it an advantage.
2. Naive Bayes underperforms** compared to other algorithms. The feature independence assumption likely does not hold for banknote features, and the Gaussian distribution may not perfectly fit the data.
3. Decision Tree with max_depth=5** achieves reasonable accuracy but shows higher variance than KNN. Pre-pruning helps prevent overfitting but may limit the model's ability to capture complex patterns.

4. **Statistical significance** is confirmed by paired t-tests. All performance differences between algorithms are statistically significant ($p < 0.05$).

5. **Learning curve analysis** reveals:

- Naive Bayes: Stable, low bias
 - KNN: Very stable, low variance, excellent even with small data
 - Decision Tree: Moderate variance, benefits from more data
-

12. Limitations of Each Algorithm

Naive Bayes

- Assumes feature independence (may not hold for banknote features)
- Gaussian distribution assumption may be violated
- Lower accuracy compared to KNN and DT

KNN

- Lazy learner - stores all training data (computationally expensive)
- Sensitive to irrelevant features
- Requires choosing optimal k
- High memory usage for large datasets

Decision Tree

- Pre-pruning with depth=5 limits complexity
 - Discretization with mean threshold loses information
 - More variance than KNN on this dataset
 - Can overfit without proper pruning
-

13. Suggestions for Improving Weakest Algorithm

Since **Naive Bayes** is the weakest algorithm on this dataset, the following improvements are suggested:

1. Try Kernel Density Estimation instead of Gaussian distribution
 2. Apply feature selection to remove redundant features
 3. Use data transformation (Box-Cox) to make features more Gaussian
 4. Consider Complement Naive Bayes for imbalanced classes
 5. Use ensemble methods (e.g., Averaged One-Dependence Estimators)
-

14. Implementation Highlights

- Zero ML library usage: All algorithms implemented from scratch
- Only allowed libraries: NumPy, pandas, matplotlib, scipy.stats
- Numerical stability: Naive Bayes uses log-probabilities
- No data leakage: Normalization parameters from training set only

- Complete evaluation: CV + t-test + learning curves

15. Output Files

All plots are saved to the **output/** folder:

File	Description
learning_curves_comparison.png	All models on one plot
algorithm_comparison.png	Bar chart with error bars
learning_curves_report.png	Clean version for report
learning_curve_naive_bayes.png	NB learning curve
learning_curve_knn_(k=X).png	KNN learning curve (best k)
learning_curve_decision_tree.png	DT learning curve

16. Conclusion

This project successfully implemented three machine learning algorithms from scratch and evaluated them on the banknote authentication dataset. KNN proved to be the most effective classifier, achieving near-perfect accuracy. The systematic evaluation using cross-validation and statistical testing provides confidence in the results. The implementation demonstrates a thorough understanding of algorithm internals and proper machine learning methodology.

17. References

1. UCI Machine Learning Repository - Banknote Authentication Dataset
 2. Tom Mitchell, "Machine Learning", McGraw-Hill, 1997
 - Chapter 1: Concept Learning (pages 1-12)
 - Chapter 3: Decision Tree Learning (pages 52-78)
 - Chapter 6: Bayesian Learning (pages 153-175)
 - Chapter 8: Instance-Based Learning (pages 230-240)
 - Chapter 5: Hypothesis Evaluation (pages 128-150)
-

18. Academic Integrity

This project is submitted as coursework. All code is original and written from scratch without using pre-built machine learning libraries.

19. Contact

For questions or clarifications regarding this implementation, please refer to the project documentation or course instructor.